

Go Fundamentals for Distributed Algorithms V

Danilo Kaćanski

Faculty of Technical Sciences, Novi Sad
Applied Algorithms in Control Systems

March 2026.

Channels

What Are Channels

- A **channel** is a **communication mechanism** between goroutines.
- Instead of sharing memory directly, goroutines **exchange data through channels**.
- This follows Go's core idea: *"Do not communicate by sharing memory, instead share memory by communicating."*

Why do we want channels?

- Reduce need for explicit locking (mutex)
- Avoid many classes of bugs like race conditions and deadlocks
- Make concurrency **structured and easier to reason about**

What channels give us?

- **Data transfer** (values move between goroutines)
- **Synchronization** (send/receive coordinate execution)
- **Decoupling** (goroutines do not need direct references to each other)

Channels vs Mutex

Shared Memory (Mutex)

- Multiple goroutines access the same data
- Use **locks** to avoid conflicts
- Risk: deadlocks, race conditions, complex reasoning

Message Passing (Channels)

- Goroutines communicate by sending values
- No shared state needed
- Easier to reason about flow of data

Key Takeaway

Mutex protects shared memory. Channels avoid sharing it in the first place.

Channels

Create, send, receive

```
func main() {  
    ch := make(chan int) // channel of int values  
    go func() {  
        ch <- 42 // send value into channel  
    }()  
    v := <-ch // receive value from channel  
    fmt.Println(v)  
}
```

- A channel is a typed communication path between goroutines created with `make(chan int)`
- `ch <- 42` sends a value, and `<-ch` receives a value.

Key Idea

Channels are used to move data and synchronize goroutines.

Unbuffered channels synchronize

```
ch := make(chan int)
go func() {
    fmt.Println("sending")
    ch <- 10 // blocks until receiver is ready
    fmt.Println("sent")
}()
fmt.Println("receiving")
v := <-ch // blocks until sender provides value
fmt.Println(v)
```

- **Send blocks:** `ch <- 10` pauses the goroutine until another goroutine executes `<-ch`.
- **Receive blocks:** `v := <-ch` pauses until some goroutine executes `ch <- value`.
- ***Receiver is ready* means:**
 - another goroutine has reached a receive operation `<-ch`
 - it is waiting (blocked) for a value

- ***Sender and receiver must meet means:***
 - both operations must be active at the same time
 - value is transferred **directly** between goroutines
 - neither side proceeds until the exchange completes

Execution Timeline

- 1 Sender reaches `ch <- 10` → waits (no receiver yet)
- 2 Main reaches `<-ch` → now both sides are ready
- 3 Value is transferred instantly
- 4 Both goroutines continue execution

Pitfall

If only one side exists (only send or only receive), the program will block forever.

Closing a Channel and comma-ok Receive

Close and detect end

```
ch := make(chan int)
go func() {
    ch <- 1
    close(ch)
}()
v1, ok1 := <-ch
v2, ok2 := <-ch
fmt.Println(v1, ok1) // 1 true
fmt.Println(v2, ok2) // 0 false
```

- `close(ch)` means: no more values will be sent.
- After close:
 - remaining buffered/sent values can still be received
 - later receives return zero value and `ok = false`

Ranging Over a Channel

Receive until closed

```
ch := make(chan int)
go func() {
    ch <- 1
    ch <- 2
    ch <- 3
    close(ch)
}()
for v := range ch {
    fmt.Println(v)
}
```

- `for v := range ch` keeps receiving until the channel is closed.
- It avoids manual repeated `<-ch` and `ok` checks.
- If the sender never closes the channel, the receiver loop may block forever.
- Use `range` when one side produces a stream and closing defines the end.

Directional Channels in Function Signatures

Read-only and write-only

```
func producer(ch chan<- int) {  
    ch <- 10  
    close(ch)  
}  
  
func consumer(ch <-chan int) {  
    v := <-ch  
    fmt.Println(v)  
}
```

- `chan<- int` means: write-only channel parameter and `<-chan int` means: read-only channel parameter.
- This improves clarity and prevent misuse.

Key Takeaway

Directional channels document intent: who sends and who receives.

Producer-Consumer with Channel Range I

Producer sends, consumer ranges

```
func producer(ch chan<- int) {  
    for i := 1; i <= 3; i++ {  
        ch <- i  
    }  
    close(ch)  
}  
  
func consumer(ch <-chan int) {  
    for v := range ch {  
        fmt.Println(v)  
    }  
}
```

- Producer sends several values, then closes the channel.
- Consumer keeps reading until the channel is closed.
- Closing lets the consumer stop cleanly without extra signals.

Producer-Consumer with Channel Range II

Pitfall

The sender usually owns channel closing. The receiver should usually not close it.

Key Takeaway

Closing a producer-owned channel is the standard way to signal “stream finished.”

Broadcast via `close(signal)` |

All goroutines wait on the same channel

```
var wg sync.WaitGroup
signal := make(chan struct{}) // empty signal channel
for i := 0; i < 5; i++ {
    wg.Add(1)
    go func(id int) {
        defer wg.Done() // mark completion when goroutine ends
        <-signal // (1) BLOCK here until signal is triggered
        fmt.Println("worker", id, "started")
    }(i)
}
close(signal) // (2) broadcast signal to ALL waiting goroutines
wg.Wait() // (3) wait for all workers to finish
```

- `<-signal` means: *wait until something happens on this channel.*
- Since nothing is ever sent, all goroutines **block at this line.**

Broadcast via `close(signal)` II

- Before `close(signal)`:
 - all goroutines are blocked at `<-signal`
 - they are waiting for a value OR channel close
- After `close(signal)`:
 - ALL goroutines unblock at the same time
 - each continues execution independently
- This acts like a **broadcast event**:
 - one action (`close`)
 - many receivers wake up

select

Receive or send

```
select {  
  case v := <-ch1:  
    fmt.Println("received from ch1:", v)  
  case ch2 <- 10:  
    fmt.Println("sent to ch2")  
}
```

- select waits until one channel operation is ready.
- Cases may be:
 - receive operations
 - send operations
- If multiple cases are ready, Go picks one.

Key Takeaway

select is the channel-based way to react to multiple possible events.

for-select

Handle whichever channel is ready

```
for i := 0; i < 6; i++ {  
    select {  
        case v := <-ch1:  
            fmt.Println("ch1:", v)  
        case v := <-ch2:  
            fmt.Println("ch2:", v)  
    }  
}
```

- A for-select loop repeatedly waits on multiple channels.
- This is useful when several producers generate values concurrently.
- The receiver processes whichever channel becomes ready first.

Key Takeaway

for-select is a standard pattern for merging multiple concurrent streams.

Optional Timeout Pattern

Stop waiting after a deadline

```
select {  
case v := <-ch:  
    fmt.Println("received:", v)  
case <-time.After(time.Second):  
    fmt.Println("timeout")  
}
```

- `time.After(d)` returns a channel that becomes ready after duration `d`.
- This is useful when a receive should not wait forever.
- Timeouts are common in concurrent and distributed systems.

Key Takeaway

Timeout channels make blocking behavior safer and more controllable.

Common Channel Pitfalls

- Forgetting to close a channel when the receiver uses `range`
- Closing a channel from the receiver side (usually wrong ownership)
- Sending on a closed channel (**panic**)
- Assuming channel operations are always non-blocking

Exercise

Task 1: Producer, Consumer, Sum

Task 1

Write a program with:

- one **producer** goroutine
- one **consumer** goroutine
- one channel of type `chan int`

The producer should:

- send numbers from 1 to N
- close the channel when done

The consumer should:

- use `for v := range ch`
- compute and print the final sum

Task 2: Broadcast with `close(signal)`

Task 2

Write a program with:

- one channel: `signal := make(chan struct{})`
- **5 worker goroutines**
- one `sync.WaitGroup`

Each worker should:

- wait on `<-signal`
- print "worker X started"

Main should:

- start all workers
- trigger them all at once using `close(signal)`
- wait for completion with `wg.Wait()`

Task 3: for-select with Timeout

Task 3

Write a program with:

- two producer goroutines
 - one sends values to `ch1`
 - one sends values to `ch2`
- main goroutine uses a **for-select** loop

Requirements:

- receive from whichever channel is ready
- print which channel the value came from
- add a timeout case using `time.After`
- when timeout happens, print "graceful stop" and exit loop